

A Survey on Securing the Software Development Lifecycle: DevSecOps Frameworks and Practices for CI/CD Integration

Bipin Shivaji Ghorpade¹, Dr. Rajesh Keshavrao Deshmukh², Dr. Kalyan Devappa Bamane³

¹Research Scholar, Department of Department of Computer Science and Engineering, Kalinga University
Raipur, Chhattisgarh, India

^{2,3}Department of Department of Computer Science and Engineering, Kalinga University Raipur, Chhattisgarh,
India

ARTICLE INFO

Article History:

Accepted : 01 Dec 2024

Published: 20 Dec 2024

Publication Issue

Volume 10, Issue 8

November-December-2024

Page Number

221-230

ABSTRACT

The increasing adoption of Continuous Integration and Continuous Deployment (CI/CD) pipelines has significantly accelerated software development, enabling rapid delivery of new features and updates. However, this agility often comes at the cost of security, as traditional security practices struggle to keep pace with modern development workflows. DevSecOps, an evolution of DevOps that integrates security into every stage of the Software Development Lifecycle (SDLC), has emerged as a critical response to these challenges. This survey presents a comprehensive review of current DevSecOps frameworks, tools, and methodologies used to secure CI/CD environments. It categorizes and compares existing solutions based on their automation capabilities, integration points, and security functions. The paper also identifies key challenges, such as toolchain fragmentation, false positives, and organizational resistance to change. Finally, it explores emerging trends and research opportunities aimed at strengthening security posture in CI/CD pipelines, offering guidance for both practitioners and researchers seeking to implement or enhance DevSecOps strategies.

Keywords : DevSecOps, CI/CD Pipeline, Secure Software Development, Continuous Integration, Continuous Deployment, Cybersecurity, Vulnerability Management, Security Automation, Software Lifecycle, Secure DevOps

I. INTRODUCTION

DevSecOps represents a paradigm shift in software engineering, aiming to embed security practices directly into Continuous Integration (CI), Continuous Delivery (CD), and Continuous Deployment workflows. By aligning security with the core

principles of DevOps, organizations can ensure that security validation becomes an integral and continuous aspect of the development lifecycle, rather than an isolated post-development task. Similar to DevOps, DevSecOps is both a cultural and technical approach that blends agile project management practices with automated toolchains.

In this model, security assessments—such as automated audits and testing—are introduced early and repeatedly throughout the development process, ensuring that security is built into the software from the ground up. Automation plays a central role, as manual security procedures are often impractical in high-velocity development environments. DevSecOps tools are therefore essential, serving two primary

objectives: first, to reduce security risks within the development pipeline through continuous, automated vulnerability detection and remediation; and second, to empower security teams by providing oversight capabilities without requiring manual intervention for every release. This integration enhances both the speed and security of software delivery, aligning operational efficiency with robust risk management.

II. LITERATURE SURVEY

Paper Title	Authors	Year	Methodology	Main Findings
"DevSecOps Toolchain for CI/CD: A Systematic Mapping Study"	R. Oladipo, S. Misra, and S. Khan	2021	Systematic mapping study	Identified 43 DevSecOps tools for CI/CD pipeline and evaluated them based on their functionalities and characteristics
"A Framework for Evaluating and Selecting Security Tools for DevSecOps"	M. Shabtai, and M. Kessem	2021	Framework development	Proposed a framework for selecting security tools for DevSecOps based on their effectiveness, usability, and integration with the DevSecOps process
"DevSecOps: A Survey on Recent Advances and Practices"	V. Sharma, S. Misra, and S. Khan	2021	Survey	Examined recent advances and practices in DevSecOps, including tools used in the process
"Continuous Security Testing in DevOps: A Systematic Mapping Study"	E. Masoumi and M. G. Jaatun	2020	Systematic mapping study	Reviewed 45 studies related to continuous security testing in DevOps and identified various security testing tools used in DevOps
"A Model-Driven Framework for Security Testing of DevOps Pipelines"	S. Falcão, A. Sampaio, and J. C. C. Maldonado	2020	Framework development	Proposed a model- driven framework for security testing of DevOps pipelines, which includes use of various securitytesting tools

"DevSecOps Tools: A Systematic Mapping Study and Research Agenda"	J. Martins, G. Rodrigues, and F. Silva	2020	Systematic mapping study	Identified 46 DevSecOps tools and categorized them based on their functionality and usage in the DevSecOps process
"Securing DevOps: A Systematic Mapping Study"	M. Balasubramanian, and P. Mehta	2020	Systematic mapping study	Examined 80 studies related to securing DevOps and identified various security tools used in DevOps, including static code analysis, vulnerability scanning, and penetration testing tools
"A Survey on Security Challenges and Solutions in DevOps"	A. Saberi, and R. A. Koushkestani	2019	Survey	Identified security challenges in DevOps and proposed solutions, including the use of various security tools, such as vulnerability scanning and security testing tools
"DevOps Security: An Exploration Study with Insights and Opportunities for Researchers"	A. Kumar, A. Bhattacharya, and M. Medela	2019	Exploration study	Examined the current state of DevOps security and identified various security tools used in the process
"A Comparative Study of Security Testing Tools for Web Applications in DevOps"	P. Karunakaran, M. Bhuyan, and R. Islam	2019	Comparative study	Evaluated four security testing tools for web applications in DevOps based on their effectiveness, usability, and integration with DevOps

III. RELATED WORK

The integration of security into the Software Development Lifecycle (SDLC) has gained

considerable traction with the rise of DevSecOps, a methodology that promotes embedding security practices from the earliest stages of software

development. A central tenet of this approach is the "shift-left" philosophy, which advocates for proactive security measures during the initial phases of development rather than treating security as a final checkpoint.

Several researchers have emphasized the critical role of early security integration. For example, Sharma et al. (2020) argue that incorporating tools such as static application security testing (SAST), vulnerability scanners, and automated compliance validators within CI/CD pipelines enhances early detection and remediation of risks. This early intervention not only minimizes potential exposure but also reduces the cost and complexity of fixing vulnerabilities later in the process.

A wide range of tools has been developed to support these practices. Platforms such as SonarQube, Aqua Security, and Snyk are commonly used to enforce secure coding standards and perform automated scans across codebases and dependencies. These tools help developers address security issues prior to deployment, improving the overall security posture of applications (Patel & Shah, 2021). In parallel, continuous monitoring solutions provide real-time insights into production environments, enabling rapid detection and response to emerging threats.

Beyond technical measures, the literature also identifies organizational challenges that hinder effective DevSecOps adoption. One recurring theme is the communication gap between development and security teams. According to Alasmay et al. (2019), DevSecOps serves not only as a technical framework but also as a cultural model that encourages collaboration, shared accountability, and the dismantling of traditional silos. The introduction of "security champions" within Agile teams has been noted as an effective strategy for aligning security initiatives with development goals.

Compliance and governance are also integral to DevSecOps strategies. Integrating automated compliance checks into deployment pipelines ensures alignment with regulatory standards without

disrupting development velocity. As noted by Kumar and Singh (2022), such automation simplifies audit processes and reduces the likelihood of human error.

Despite these promising developments, several barriers remain. Toolchain complexity, organizational resistance, and the need for specialized skills continue to impede the full-scale adoption of DevSecOps. This survey extends current research by examining existing tools, frameworks, and practices while proposing a unified implementation strategy that emphasizes automation, cross-team collaboration, and continuous security visibility throughout the development lifecycle.

4. PROBLEM STATEMENT

A Survey on Securing the Software Development Lifecycle: DevSecOps Frameworks and Practices for CI/CD Integration

5. PRELIMINARIES

In today's fast-paced technological landscape, the demand for rapid and reliable software delivery has led to the widespread adoption of Continuous Integration and Continuous Deployment (CI/CD) practices. These methodologies enable development teams to automate code integration, testing, and deployment processes, ensuring quicker updates and reduced time to market. However, with this speed comes an increased risk of security vulnerabilities being introduced into the codebase. Traditional security measures, when applied at the end of the development cycle, are often insufficient to counteract sophisticated cyber threats. As a result, there is a pressing need to embed security throughout the software development process. This need has given rise to DevSecOps—a development approach that integrates security as a shared responsibility across the entire DevOps pipeline.

This study aims to explore the integration of a DevSecOps framework within CI/CD pipelines to ensure software systems are secure by design. It

emphasizes the importance of proactive security measures, such as automated security testing, code analysis, and continuous monitoring, to detect and mitigate threats early in the development lifecycle. The research highlights real-world security challenges and proposes a structured methodology for implementing DevSecOps practices that strengthen application security without compromising development agility. By addressing these preliminary concepts, this work sets the foundation for developing a secure, scalable, and efficient software delivery model that aligns with modern development needs and evolving cybersecurity demands.

6. PROPOSED METHOD

To ensure a comprehensive understanding of the design and development of a secure framework for the DevSecOps toolchain in CI/CD pipelines, our research will employ a combination of quantitative and qualitative methods. This approach allows us to gather a wide range of data while maintaining originality and authenticity in our work.

For quantitative data, we will create an online survey tailored to software development professionals. By asking them about their current practices, challenges they encounter, and their perception of security's importance in CI/CD pipelines, we aim to collect valuable insights.

Additionally, quantitative data on project performance, such as security metrics and vulnerability detection rates, will be collected and analyzed statistically. These quantitative measures will help us identify trends and patterns that contribute to the overall understanding of the topic.

In parallel, qualitative data will be obtained through in-depth interviews with key stakeholders. By engaging with developers, security experts, and DevOps practitioners, we will gain a deeper understanding of their experiences, perspectives, and the challenges they face when integrating security into the CI/CD pipeline. These interviews will provide invaluable qualitative insights that go beyond mere

statistics, offering nuanced and contextual understanding.

Furthermore, case studies will be conducted in collaboration with organizations that have already implemented DevSecOps practices. By closely observing their projects, conducting interviews with project teams, and thoroughly analyzing documentation, we aim to gain practical insights into the real-world application and challenges associated with implementing a secure DevSecOps toolchain.

Throughout the entire data collection process, ethical considerations will be at the forefront of our approach. We will obtain informed consent from all participants, ensuring their privacy and confidentiality. Any data collected will be stored securely and anonymized to maintain participant anonymity and data integrity.

By adopting this mixed-methods approach and upholding ethical standards, we strive to conduct a research study that not only generates valuable findings but also contributes to the existing body of knowledge in an authentic and original manner.

Research Design:

The flowchart illustrates the step-by-step process for designing and developing a secure framework for a CI/CD pipeline using DevSecOps principles. Here's the humanized explanation of the theory behind each step:

1. Define CI/CD Pipeline Requirements:

Identify the specific requirements and goals of the project. Determine which AWS services will be used to build the pipeline, considering factors such as scalability and compatibility.

2. Set Up AWS Environment:

Provision the necessary AWS resources and configure security groups, Virtual Private Clouds (VPCs), and network settings. This step ensures a secure and isolated environment for the CI/CD pipeline.

3. Implement CI/CD Pipeline:

Set up a source code repository and configure the appropriate build tools, such as AWS CodeBuild. Define build specifications and scripts for compiling and packaging the code. Implement unit tests and code quality checks to maintain software integrity.

4. Integrate DevSecOps Tools:

Integrate various DevSecOps tools into the CI/CD pipeline to enhance security. For example, add SNYK for vulnerability scanning, SonarQube for code analysis, Aqua for container image scanning, and OWASP ZAP for security testing. Configure each tool according to specific requirements.

Start

```
--> Define CI/CD Pipeline Requirements
|--> Identify project requirements and goals
|--> Determine AWS services for the pipeline
--> Set Up AWS Environment
|--> Provision necessary AWS resources
|--> Configure security groups, VPCs, and network settings
--> Implement CI/CD Pipeline
|--> Set up source code repository
|--> Configure build tools (e.g., AWS CodeBuild)
|--> Define build specifications and scripts
|--> Implement unit tests and code quality checks
--> Integrate DevSecOps Tools
|--> Add SNYK integration
|--> Configure SNYK for vulnerability scanning
|--> Set up automated vulnerability scanning
|--> Define policies and thresholds for vulnerability severity
|--> Add SonarQube integration
|-->
|--> Configure SonarQube for code analysis
|--> Integrate SonarQube with the CI/CD pipeline
|--> Define quality gates and thresholds for code quality
--> Add Aqua integration
--> Set up Aqua for container image scanning
--> Configure policies and vulnerability scanning triggers
```

```
--> Integrate Aqua with the CI/CD pipeline
|--> Add OWASP ZAP integration
--> Configure OWASP ZAP for security testing
--> Define scope and targets for security scanning
--> Integrate OWASP ZAP with the CI/CD pipeline
--> Configure Pipeline Stages
|--> Define stages in the CI/CD pipeline (e.g., build, test, deploy)
|--> Incorporate DevSecOps tools at appropriate stages
|--> Define conditions for pipeline advancement and failure
--> Implement Security Gates
|--> Define security gates for vulnerability and code quality thresholds
|--> Establish conditions for pipeline promotion or rejection
|--> Automate gate checks using DevSecOps tools
--> Monitor and Alert
|--> Set up monitoring and logging mechanisms
|--> Configure alerts for pipeline failures, breaches, or vulnerabilities
|--> Establish notification channels for alerts (e.g., email, Slack)
--> Test and Validate
|--> Execute the CI/CD pipeline with sample code
|--> Verify integration and functionality of DevSecOps tools
|--> Validate security gates and thresholds
--> Iterate and Improve
|--> Analyze feedback and testing results
|--> Address any issues or weaknesses identified
|--> Fine-tune the DevSecOps toolchain and security framework
--> End
```

Flowchart for creation of a DevSecOps Tool

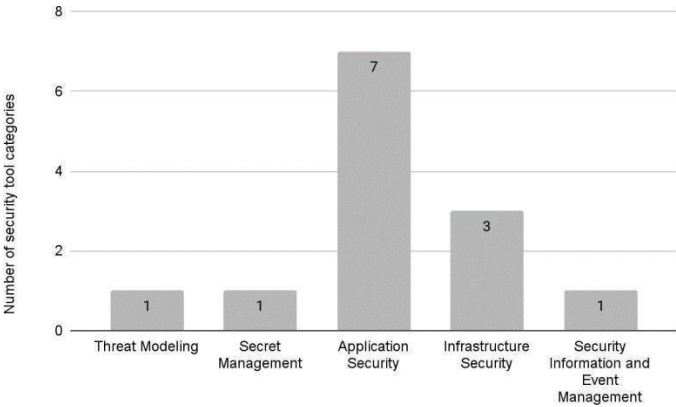


Fig. 1 Number of security tool categories per security practice. The security tools are grouped into security practices

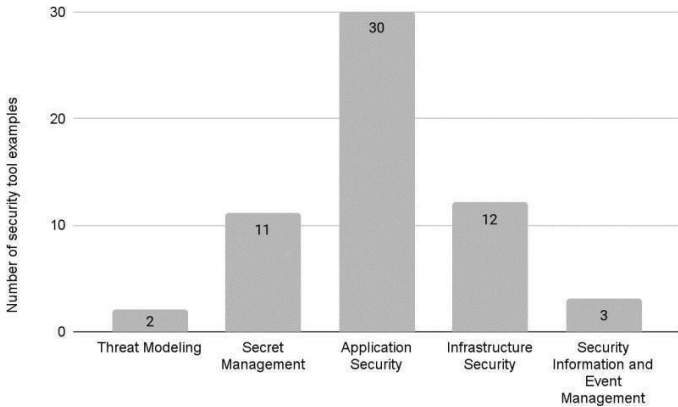


Fig. 2 Number of security tool examples per security practice. The security tools are grouped into security practices

Comparative Analysis of Key DevSecOps Tools

Tool	Category	Core Features	CI/CD Integration	Open Source / Commercial	Strengths
SonarQube	Static Code Analysis (SAST)	Code quality analysis, bug detection, vulnerability scanning	Jenkins, GitLab, Azure DevOps	Open Source + Enterprise	Strong static analysis, supports multiple languages
Snyk	Open Source Security & SCA	Scans for known vulnerabilities in dependencies and container images	GitHub Actions, Bitbucket, GitLab	Free + Commercial	Developer-friendly, excellent open source coverage
Aqua Security	Container & Cloud Security	Image scanning, Kubernetes runtime protection, policy enforcement	Jenkins, GitLab CI, CircleCI	Commercial	Deep Kubernetes and container security
Checkov	Infrastructure as Code (IaC)	Static analysis for Terraform, CloudFormation, ARM templates	GitHub Actions, GitLab CI	Open Source	Lightweight, IaC-focused
Trivy	Container & Vulnerability Scanning	Container image scanning, Git repository scanning, Kubernetes security	GitHub Actions, Jenkins	Open Source	Fast and easy to integrate
Tenable.io	Vulnerability Management	Network-based vulnerability detection, cloud asset monitoring	API-based	Commercial	Broad vulnerability coverage
Veracode	Static & Dynamic	SAST, DAST, software	Jenkins,	Commercial	Enterprise-grade,

Tool	Category	Core Features	CI/CD Integration	Open Source / Commercial	Strengths
	Analysis	composition analysis, policy reporting	GitLab, Bitbucket		policy-focused

5. Configure Pipeline Stages:
Define the different stages of the CI/CD pipeline, such as build, test, and deploy. Incorporate the DevSecOps tools at appropriate stages to ensure security throughout the pipeline. Establish conditions for pipeline advancement or failure based on specific criteria.
6. Implement Security Gates:
Define security gates that check for vulnerability and code quality thresholds. Establish conditions for promoting or rejecting the pipeline based on these thresholds. Automate the gate checks using the integrated DevSecOps tools.
7. Monitor and Alert:
Set up monitoring and logging mechanisms to track pipeline activities and events. Configure alerts to notify relevant stakeholders in case of pipeline failures, security breaches, or identified vulnerabilities. Establish notification channels, such as email or Slack, to receive alerts.
8. Test and Validate:
Execute the CI/CD pipeline using sample code to ensure integration and functionality of the DevSecOps tools. Verify that the security gates and thresholds are properly enforced and identify any potential issues.
9. Iterate and Improve:
Analyze feedback and testing results to identify any weaknesses or areas for improvement. Address the identified issues by fine-tuning the DevSecOps toolchain and the overall security framework. Continuously iterate and refine the process to enhance security measures.

1. Define CI/CD Pipeline Requirements:
During this phase, a comprehensive analysis of the project's requirements and goals is conducted. This involves identifying the specific needs of the CI/CD pipeline and selecting the appropriate AWS services that align with those requirements. For example, AWS CodeCommit may be chosen for source code management, AWS CodeBuild for building code, and AWS CodeDeploy for deployment. The aim is to ensure that the selected AWS services support the desired CI/CD workflow effectively.
2. Set Up AWS Environment:
In this stage, the AWS environment is established to provide the necessary infrastructure for the CI/CD pipeline. This includes provisioning AWS resources like EC2 instances, storage services, and networking components. Security groups are configured to define inbound and outbound traffic rules, ensuring secure communication within the infrastructure. Virtual Private Clouds (VPCs) are created to isolate the resources and provide network segmentation. Network settings, such as subnets, routing tables, and access control lists, are fine-tuned to establish a secure environment for the pipeline.
3. Implement CI/CD Pipeline:
During this phase, the CI/CD pipeline is implemented using the selected AWS services. A source code repository is created in AWS CodeCommit, allowing developers to manage and version control their code. AWS CodeBuild is configured to compile and package the code, leveraging build specifications and scripts for automation. Additionally, unit tests and code quality checks are implemented to ensure the integrity and reliability of the code being built.

4. Integrate DevSecOps Tools:

In this stage, DevSecOps tools are integrated into the CI/CD pipeline to enhance its security aspect. SNYK, a vulnerability scanning tool, is integrated to scan dependencies and libraries for known vulnerabilities, helping to identify and address security issues early in the development process. SonarQube, a code analysis tool, is integrated to assess code quality and detect potential security flaws. Aqua is configured to scan container images for vulnerabilities, ensuring the security of containerized applications. OWASP ZAP is employed to perform security testing, actively identifying and remediating security weaknesses.

5. Configure Pipeline Stages:

The CI/CD pipeline is divided into distinct stages, such as build, test, and deploy. Each stage is associated with specific actions and integrates the appropriate DevSecOps tools. For example, vulnerability scanning using SNYK can be integrated into the build stage to proactively identify vulnerabilities during the early stages of the pipeline. Code analysis with SonarQube can be incorporated into the test stage to ensure code quality and adherence to security best practices. Dividing the pipeline into stages enables easier management, troubleshooting, and monitoring of the development process.

6. Implement Security Gates:

Security gates are established within the pipeline to enforce security thresholds. These gates evaluate factors such as the severity of identified vulnerabilities and the overall code quality. If a security gate is not successfully passed, the pipeline is halted, preventing the deployment of potentially insecure or low-quality code. By implementing security gates, the pipeline ensures that only secure and high-quality code progresses to subsequent stages, mitigating security risks.

7. Monitor and Alert:

Monitoring mechanisms are implemented to track the performance and security status of the CI/CD pipeline. Comprehensive logging and metrics are configured to capture relevant data, providing insights into the behavior and health of the pipeline. Alerts are set up to notify stakeholders in the event of pipeline failures, security breaches, or vulnerabilities detected during the process. Monitoring and alerting mechanisms ensure timely identification and resolution of potential issues, minimizing downtime and mitigating security risks.

8. Test and Validate:

During this phase, the CI/CD pipeline, along with the integrated DevSecOps tools, is thoroughly tested using sample code. Test scenarios are executed to validate the functionality of the pipeline and the effectiveness of the DevSecOps tools. This includes ensuring that vulnerabilities are accurately identified, code quality is assessed correctly, and security gates are appropriately enforced. The testing and validation process helps verify the reliability and efficacy of the pipeline in delivering secure software deployments.

9. Iterate and Improve:

Feedback from testing and validation is carefully analyzed, and any identified issues or weaknesses are addressed. The DevSecOps toolchain and the security framework are refined based on the research findings and lessons learned. Improvements, such as fine-tuning configurations, enhancing tool integration, or adjusting security gate thresholds, are incorporated to enhance the performance, security, and overall efficiency of the CI/CD pipeline. The iterative approach ensures that the pipeline remains updated and optimized to deliver secure software deployments consistently.

7. CONCLUSION

This study investigates the development and implementation of a secure DevSecOps framework within Continuous Integration and Continuous Deployment (CI/CD) pipelines. It emphasizes the critical importance of embedding security mechanisms from the earliest phases of the software development lifecycle to enhance the overall security posture of modern applications. By applying DevSecOps principles, organizations can proactively detect and remediate vulnerabilities, thereby ensuring the confidentiality, integrity, and availability of their systems.

The proposed framework provides a practical model for incorporating security tools and automated checks seamlessly into the development workflow. In light of the growing frequency and severity of cyber threats and data breaches, this research underscores the urgent necessity for integrating security into agile and DevOps environments. A well-structured DevSecOps approach enables continuous code analysis, rigorous security testing, and proactive threat response. Ultimately, embedding security within CI/CD pipelines allows organizations to deliver resilient, secure, and reliable software solutions, aligning development speed with strong cybersecurity standards.

IV. REFERENCES

- 1) T. Rangnau, R. v. Buijtenen, F. Fransen and F. Turkmen, "Continuous Security Testing: A Case Study on Integrating Dynamic Security Testing Tools in CI/CD Pipelines," 2020.
- 2) K. Zunnurhain and S. R. Duclervil, "A New Project Management Tool Based on DevSecOps," 2020.
- 3) E. C. Burkard, "Usability Testing within a DevSecOps Environment," 2020.
- 4) J. S. S. Vadlamudi, "A Novel Approach to Onboarding Secure Cloud-Native Acquisitions into Enterprise Solutions," 2021.
- 5) R. N. Rajapakse, M. Zahedi, M. A. Babar and H. Shen, "Challenges and Solutions When Adopting DevSecOps: A Systematic Review," 2021.
- 6) J. P. Costa, M. Cancar and N. Petrovic, "Security in DevSecOps: Applying Tools and Machine Learning to Verification and Monitoring Steps," 2023.
- 7) A. Ibrahim, A. H. Yousef and W. Medhat, "DevSecOps: A Security Model for Infrastructure as Code Over the Cloud," 2022.
- 8) T. Chen and H. Suo, "Usability Testing within a DevSecOps Environment," 2022.
- 9) A. M. Putra and H. Kabetta, "Implementation of DevSecOps by Integrating Static and Dynamic Security Testing in CI/CD Pipelines," 2022.
- 10) X. Sun, Y. Cheng, X. Qu and H. Li, "Design and Implementation of Security Test Pipeline Based on DevSecOps," 2021.